



C++ PROGRAMMING

(335)

STATE 2023

APPLICATION KNOWLEDGE:

Payroll Calculator and Statistics _____ (400 points)

TOTAL POINTS _____ (400 points)

Test Time: 90 minutes

Payroll Calculator and Statistics

In the modern workplace, accounting applications are widely used to assist with hours tracking and payroll calculation. Increasingly, they are also being used to track variations in employee behavior as detected by their hours (among other things). In this exercise, you will create a C++ console application that performs this functionality.

The data for this application will be stored in a simple text file, with each 5 lines being related to a particular employee. The first line will contain the employee's name, the hourly pay rate, and the modifier applied for overtime hours (applied to any hours over 40 in a week).

The next lines will indicate the numbers of hours worked that week, comma separated, starting with Sunday, and ending on Saturday. So, for the first week listed for Figure 1, Edward Saar worked 0 hours on Sunday, Friday, and Saturday, and 10 hours all other days.

The first two weeks of hour data are for the two preceding weeks, while the last two weeks of hour data are for the current pay period – doing so, those statistics can be reported, and variations can be evident.

```
Edward Saar;50.35;1.5  
0,10,10,10,10,0,0  
0,10,10,10,10,5,0  
0,10,10,10,10,5,0  
0,10,10,10,10,0,0
```

Figure 1: Test input example

Once the data is loaded, calculations related to payroll, hours worked (current and previous period, and average), and more importantly hour variations can be reported in a manner shown below.

	NAME	PAY_RATE	OT_BONUS	CUR_PAY	CUR_HOUR	PRV_PAY	PRV_HOUR	HOUR_DIFF	AVG_PAY	AVG_HOUR
Edgar	Edward Saar	50.35	1.50	4405.62	85	4405.62	85	0	4405.62	85
	Brad North	20.10	2.00	1045.20	52	1809.00	85	-33	1427.10	68
	Skarsgard	40.80	3.00	3060.00	55	1509.60	37	18	2284.80	46
	Zach Kolluri	27.95	1.50	1956.50	70	1788.80	64	6	1872.65	67
	Pete Bert	25.35	2.00	3650.40	112	3650.40	112	0	3650.40	112
	Fredric Watt	36.35	2.50	2998.88	75	1635.75	45	30	2317.31	60
	Dennis Pasko	14.35	1.00	1004.50	70	1004.50	70	0	1004.50	70
Total Hours Current Period:				519						
Total Hours Previous Period:				498						
Average Hours Per Employee:				71						
Total Payment Current Period:				18121.10						
Total Payment Previous Period:				15803.67						
Average Payment Per Employee:				2257.67						
Employee with lowest hours this period:				Brad North						
Employee with highest hours this period:				Pete Bert						
Press any key to continue . . .										

Figure 2: Sample output for provided input – note that tested input file may be different (do not hardcode report)

Requirements:

1. You must create a C++ console application named CPP_335_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CPP_335_01_2345_6789.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. The Application shall include a type for the purpose of storing a list of objects of a data type employee
 - a. The details of what manner of type for either the list or the employee is up to the contestant, but it must be a logical use
 - b. The type for employee shall have a function for the following items
 - i. Retrieving average pay and hours
 - ii. Retrieving normal hours (defined as any number of hours worked in a week up to the overtime level) and overtime hours (defined as any number of hours worked over 40, the overtime level)
 - iii. Retrieve number of hours worked in the current and previous pay period
 - iv. Retrieving the name of the employee
4. The file shall be loaded on application started, parsed, and the report displayed generated, prior to pausing the application and then closing
 - a. The loading and parsing of the file shall be performed in the manner described, retrieving data for each of the weeks individually
 - b. Proper error handling shall be applied for reading through the file
5. Functions shall be constructed to perform the following actions on the list of employees
 - a. Creating the employee report table
 - i. This shall be formatted in the manner shown
 - b. Performing averaging and total hours and pay calculations, followed by a simple report
 - c. Reporting the employees who worked the least and most hours

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 pts

The project is named according to the naming conventions _____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB _____ 10 pts

If the program does not execute, then the remaining items in this section receive a score of zero.

Application loads file on startup (and appropriately errors if not present) _____ 20 pts

Application generates proper and formatted employee overview _____ 60 pts

Application generates proper total and average report _____ 40 pts

Application generates proper min/max hour report _____ 30 pts

Application pauses prior to exiting _____ 20 pts

Source Code Review

Code is commented at the top, for each function, and as needed _____ 20 pts

Code uses reasonable and consistent variable naming conventions _____ 20 pts

Employee type exists and has suitable functions and constructors _____ 50 pts

Employee list type exists, and functions as described _____ 20 pts

File/error handling and parsing is handled as described _____ 30 pts

Total Points: 350 pts

Completed Source Code

```

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <iomanip>
#include <sstream> // Allows splitting of string

using namespace std;

const string filePath = "hours.txt";

// Splits a selected string into a collection
// of strings separated by a given delimiter
vector<string> Split(char delim, string input)
{
    vector<string> tokens; // Collection of string tokens
    stringstream stream(input); // Stringstream of input
    string token; // Used to pull out individual tokens
    // While delims still present in string...
    while (getline(stream, token, delim))
        if (token.compare(""))
            tokens.push_back(token); // Add to vector
    return tokens; // Returns collection
}

typedef struct emp
{
    string name = "";
    double pay = 0.0;
    double otbonus = 0.0;
    int hoursWeek1 = 0;
    int hoursWeek2 = 0;
    int hoursWeek3 = 0;
    int hoursWeek4 = 0;

    string getName()
    {
        return name;
    }

    int getHoursCurPeriod()
    {
        return hoursWeek3 + hoursWeek4;
    }

    int getHoursPrvPeriod()
    {
        return hoursWeek1 + hoursWeek2;
    }

    static int getOtHours(int hours)
    {
        return hours > 40 ? hours - 40 : 0;
    }

    static int getNormHours(int hours)
    {

```

```

        return hours > 40 ? 40 : hours;
    }

    static double getPay(int hours, double pay, double otbonus)
    {
        return (static_cast<double>(getNormHours(hours)) * pay)
            + (static_cast<double>(getOtHours(hours)) * pay * otbonus);
    }

    double getPayCurPeriod()
    {
        return getPay(hoursWeek3, pay, otbonus)
            + getPay(hoursWeek4, pay, otbonus);
    }

    double getPayPrvPeriod()
    {
        return getPay(hoursWeek1, pay, otbonus)
            + getPay(hoursWeek2, pay, otbonus);
    }

    int getAvgHours()
    {
        return (getHoursCurPeriod() + getHoursPrvPeriod()) / 2;
    }

    double getAvgPay()
    {
        return (getPayCurPeriod() + getPayPrvPeriod()) / 2.0;
    }
} employee;

typedef vector<employee> employeeList;

void parseHours(ifstream& stream, bool& fail, int& hours)
{
    string val;

    if (getline(stream, val))
    {
        vector<string> hourtoken = Split(',', val);
        if (hourtoken.size() == 7)
        {
            for (int i = 0; i < 7; i++)
            {
                hours += stoi(hourtoken[i]);
            }
        }
        else
            fail = true;
    }
    else
        fail = true;
}

employee fromFile(ifstream& stream, bool& fail)
{
    employee res;
    string val;

```

```

    if (getline(stream, val))
    {
        vector<string> nametoken = Split(';', val);
        if (nametoken.size() == 3)
        {
            res.name = nametoken[0];
            res.pay = stod(nametoken[1]);
            res.otbonus = stod(nametoken[2]);

            parseHours(stream, fail, res.hoursWeek1);
            if (!fail)
                parseHours(stream, fail, res.hoursWeek2);
            if (!fail)
                parseHours(stream, fail, res.hoursWeek3);
            if (!fail)
                parseHours(stream, fail, res.hoursWeek4);
        }
        else
            fail = true; // incorrect token count
    }
    else
        fail = true; // no line present

    return res;
}

employeeList retrieveEmployees()
{
    employeeList employees;

    ifstream dataFile(filePath); // Creates ifstream object
    if (dataFile.is_open()) // Ensures file is open
    {
        string line;
        bool fail = false;

        do
        {
            employee res = fromFile(dataFile, fail);
            if (!fail)
                employees.push_back(res);
        } while (!fail);
    }
    else
    {
        cout << "Failed to open file at following location: " << filePath << endl;
    }

    return employees;
}

void repExtremes(employeeList employees)
{
    int idxMaxHours = 0, idxMinHours = 0;
    for (int i = 0; i < employees.size(); i++)
    {
        employee e = employees[i];
        if (e.getHoursCurPeriod() > employees[idxMaxHours].getHoursCurPeriod())

```

```

        {
            idxMaxHours = i;
        }
        if (e.getHoursCurPeriod() < employees[idxMinHours].getHoursCurPeriod())
        {
            idxMinHours = i;
        }
    }

    cout << "Employee with lowest hours this period:  " << left <<
employees[idxMinHours].getName() << endl;
    cout << "Employee with highest hours this period:  " <<
employees[idxMaxHours].getName() << endl;

    cout << endl;
}

void repTotalAvg(employeeList employees)
{
    int totalHoursCurPeriod = 0;
    int totalHoursPrvPeriod = 0;
    double totalPaidCurPeriod = 0.0;
    double totalPaidPrvPeriod = 0.0;

    for (int i = 0; i < employees.size(); i++)
    {
        employee e = employees[i];

        totalHoursCurPeriod += e.getHoursCurPeriod();
        totalHoursPrvPeriod += e.getHoursPrvPeriod();
        totalPaidCurPeriod += e.getPayCurPeriod();
        totalPaidPrvPeriod += e.getPayPrvPeriod();
    }

    cout << "Total Hours Current Period:          " << right << setw(10) <<
totalHoursCurPeriod << endl;
    cout << "Total Hours Previous Period:          " << setw(10) << totalHoursPrvPeriod <<
endl;
    cout << "Average Hours Per Employee:          " << setw(10) << totalHoursPrvPeriod /
employees.size() << endl;
    cout << "Total Payment Current Period:          " << setw(10) << totalPaidCurPeriod <<
endl;
    cout << "Total Payment Previous Period:          " << setw(10) << totalPaidPrvPeriod <<
endl;
    cout << "Average Payment Per Employee:          " << setw(10) << totalPaidPrvPeriod /
employees.size() << endl;

    cout << endl;
}

void repOverview(employeeList employees)
{
    cout << right << setw(15) << "NAME" <<
        setw(10) << "PAY_RATE" <<
        setw(10) << "OT_BONUS" <<
        setw(15) << "CUR_PAY" <<
        setw(10) << "CUR_HOUR" <<
        setw(15) << "PRV_PAY" <<
        setw(10) << "PRV_HOUR" <<

```

```
        setw(10) << "HOUR_DIFF" <<
        setw(15) << "AVG_PAY" <<
        setw(10) << "AVG_HOUR" << endl;

    for (int i = 0; i < employees.size(); i++)
    {
        employee e = employees[i];
        cout <<
            setw(15) << e.name <<
            fixed << setprecision(2) << setw(10) << e.pay <<
            setw(10) << e.otbonus <<
            setw(15) << e.getPayCurPeriod() <<
            setw(10) << e.getHoursCurPeriod() <<
            setw(15) << e.getPayPrvPeriod() <<
            setw(10) << e.getHoursPrvPeriod() <<
            setw(10) << e.getHoursCurPeriod() - e.getHoursPrvPeriod() <<
            setw(15) << e.getAvgPay() <<
            setw(10) << e.getAvgHours() <<
            endl;
    }

    cout << endl;
}

int main()
{
    employeeList employees = retrieveEmployees();

    repOverview(employees);
    repTotalAvg(employees);
    repExtremes(employees);

    system("pause");
    return 0;
}
```